

StudyAI – Smart AI educate Companion & Quiz Generator

Project Description:

StudyAI – Smart AI Educate Companion & Quiz Generator is an AI-powered web application designed to enhance students' learning experiences. The platform allows users to upload study materials such as PDFs, documents, and text files and automatically generates summaries, flashcards, and quizzes. It provides a personalized and interactive learning environment that makes studying more efficient and engaging.

The project is developed using React, Supabase, and AI APIs to deliver intelligent educational features and real-time performance tracking. It also includes a personalized study planner and analytics dashboard to monitor learning progress and identify weak areas. StudyAI aims to simplify exam preparation and support self-paced learning through advanced artificial intelligence technologies.

Scenarios

Scenario 1: A student uploads a PDF chapter, DOCX file, or pastes raw text into the upload interface. The system extracts the content, stores the material with a unique ID, and makes it available for all AI-powered processing features including summary generation, flashcard creation, and quiz generation.

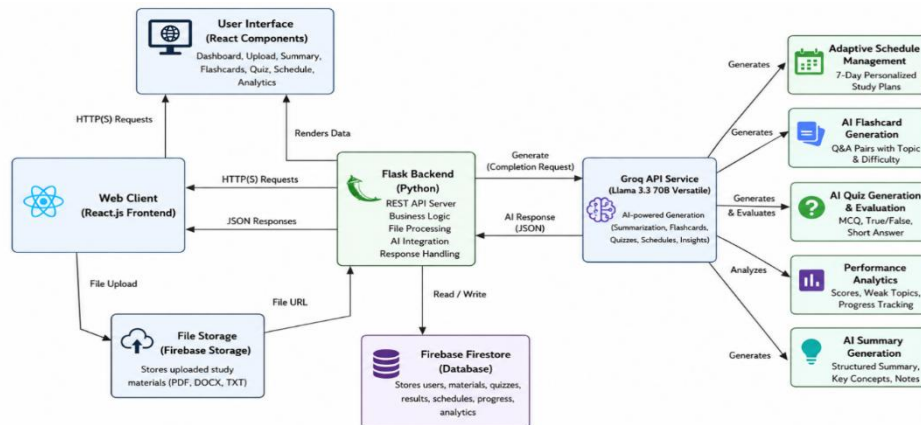
Scenario 2: A student generates a summary from their uploaded material. The Groq-powered AI reads the full content and returns a structured markdown summary with a topic overview, key concepts, important definitions, core principles, and bullet-point revision notes for last-minute review.

Scenario 3: A student creates a set of flashcards from their study material. The AI generates question-and-answer pairs with topic labels and difficulty levels. The student flips through cards, marks known cards, shuffles the deck, and tracks how many concepts they have mastered.

Scenario 4: A student selects a quiz type (MCQ, True/False, or Short Answer), chooses the number of questions, and takes an AI-generated quiz. After submission, the system calculates the score, displays correct answers with explanations, identifies weak topics, and saves performance data to the dashboard.

Scenario 5: The student opens the Schedule page to generate a personalized 7-day study plan. The AI uses the uploaded material content and any weak topics identified from quiz results to create a day-by-day schedule with tasks, time slots, focus topics, and study tips.

Technical Architecture:



Description: StudyAI – Smart AI Educate Companion & Quiz Generator is an AI-powered learning platform that helps students study more effectively. It allows users to upload study materials such as PDFs, documents, and text files and automatically generates summaries, flashcards, and quizzes. The system also provides personalized study schedules and progress analytics to improve learning outcomes. Built using React, Supabase, and AI APIs, the platform offers an interactive and user-friendly learning experience. StudyAI aims to simplify exam preparation and promote self-paced, intelligent learning.

(Note: If you are using database in your project definitely you need to add ER Diagram along with description)

Pre-requisites:

- **Flask Framework Knowledge:** [Flask Documentation](#)
- **Groq API Familiarity:** [Groq API Documentation](#)
- **HTML, CSS, and JavaScript Skills:** [W3Schools HTML/CSS/JavaScript Tutorials](#)
- **Python Programming Proficiency:** [Python Documentation](#)
- **Version Control with Git:** [Git Documentation](#)
- **Development Environment Setup:** [Flask Installation Guide](#)
- **Ngrok for Public Deployment:** [Ngrok Documentation](#)

Project Workflow:

Milestone 1: Model Selection and Architecture

- **Activity 1.1:** Generate a Groq API key and configure it securely in the project environment.
- **Activity 1.2:** Research and select the Llama 3.3 70B Versatile model from Groq for AI-powered summarization, quiz generation, flashcard creation, and study plan generation.
- **Activity 1.3:** Define the architecture of the application, detailing interactions between the React frontend, Flask backend, Supabase database, and AI integration.
- **Activity 1.4:** Set up the development environment, installing necessary libraries and dependencies for React.js, Flask, Supabase, and the Groq API.

Milestone 2: Core Functionalities Development

- **Activity 2.1:** Develop the core functionalities: Document Upload, AI Summary Generator, Flashcard Generator, Quiz Generator, and Personalized Study Schedule Generator.
- **Activity 2.2:** Implement the Flask backend to manage routing, document processing, database operations, and AI API interactions.

Milestone 3: App.py Development

- **Activity 3.1:** Write the main application logic in app.py, establishing routes for document upload, summary generation, flashcard creation, quiz generation, and study schedule generation while integrating AI responses.

Milestone 4: Frontend Development

- **Activity 4.1:** Design and develop the user interface using React.js, HTML, CSS, and JavaScript, ensuring a responsive and intuitive dashboard layout.
- **Activity 4.2:** Create dynamic React components and pages to display uploaded materials, summaries, flashcards, quizzes, analytics, and personalized study schedules.

Milestone 5: Deployment

- **Activity 5.1:** Prepare the application for deployment by configuring environment variables, database connections, and ensuring all dependencies are correctly installed.
- **Activity 5.2:** Deploy the frontend using Vercel and the Flask backend using Render/Koyeb, making the application publicly accessible.

Milestone 6: Conclusion

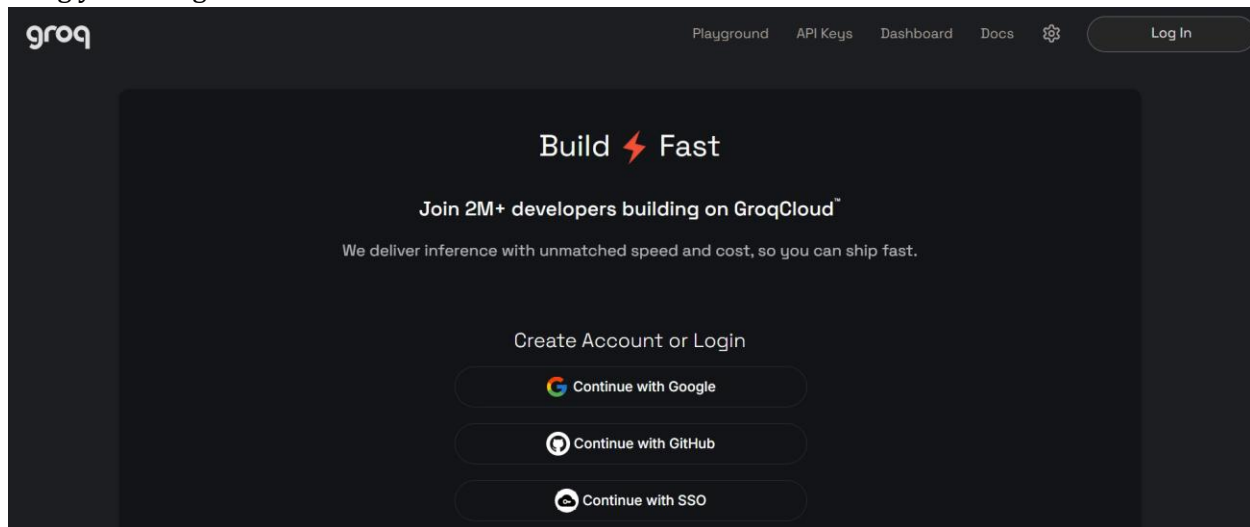
Milestone 1: Model Selection and Architecture

In this milestone, we focus on selecting the appropriate generative AI model from Groq for our social media content generation needs. This involves researching the capabilities and performance of various models that Groq offers, ensuring that the chosen model aligns well with the application's objectives of generating captions, hashtags, and CTAs across different platforms and tones.

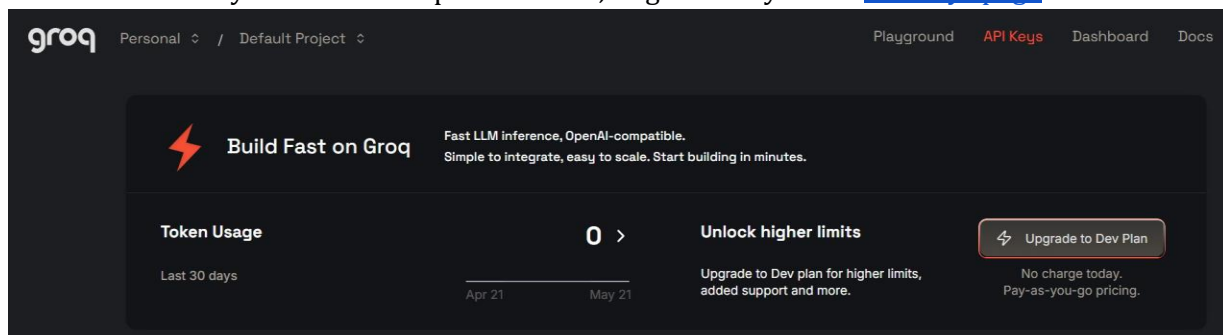
Activity 1.1: Generate a Groq API Key

Before the application can communicate with the Groq LLM, you need a valid Groq API key. Follow the steps below to create one and connect it securely to the project.

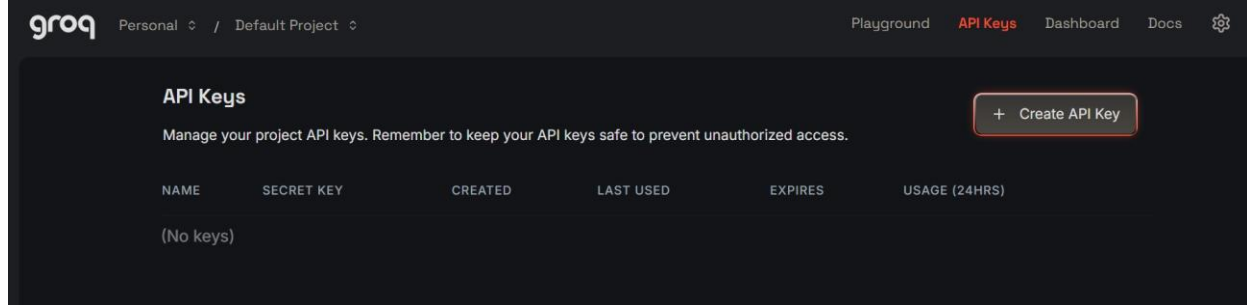
- **Step 1 — Create a Groq Account:** Visit <https://console.groq.com> and sign up for a free account using your Google account or email address.



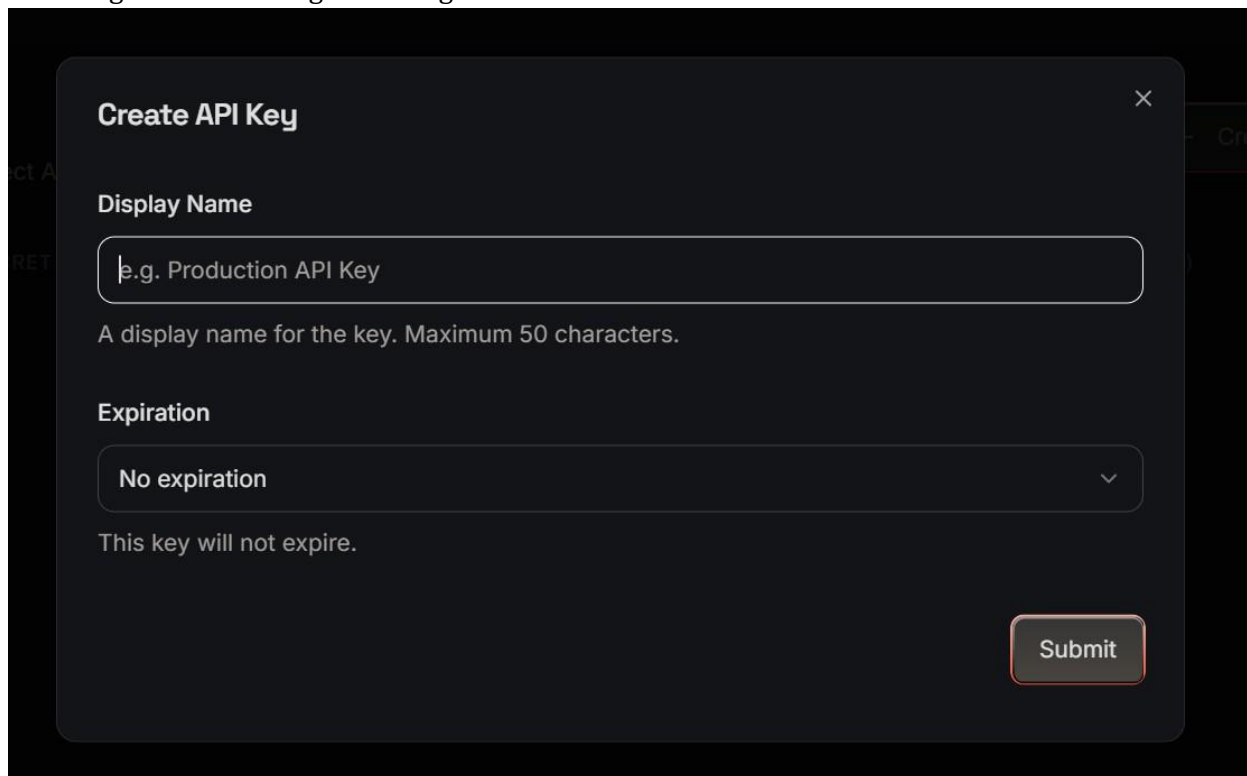
- **Step 2 — Navigate to API Keys:** After logging in, click on your profile icon in the top-right corner and select “API Keys” from the dropdown menu, or go directly to the [API Keys page](#).



- **Step 3 — Create a New API Key:** Click the “Create API Key” button. Give your key a descriptive name (e.g., captionai-key) so it is easy to identify later.



- **Step 4 — API Key:** Now give a “Display Name” and “Expiration” is optional, and Click on “Submit”. The you will get your API key, Copy it immediately and store it in a safe place you will not be able to view it again after closing the dialog.



- **Step 5 — Add the Key to Your Project:** Create a .env file in the root of your project directory (if it does not already exist) and paste the key as shown below:

GROQ_API_KEY=your_copied_api_key_here

- **Step 6 — Verify the Key is Loaded:** The app.py file loads this key automatically at startup using python-dotenv:

```
from dotenv import load_dotenv
load_dotenv()
client = Groq(api_key=os.environ.get("GROQ_API_KEY"))
```

- **Important — Never Commit Your API Key:** Add .env to your .gitignore file to prevent accidentally pushing the key to a public repository:

.env

Activity 1.2: Research and Select the Appropriate Generative AI Model

Here are the things that needed to research to select the best model for the project:

- **Understand the Project Requirements:** Review the specific needs of the StudyAI application, focusing on AI-powered features such as document summarization, flashcard generation, quiz creation, and personalized study schedule generation.
- **Explore Groq's Model Documentation:** Visit the Groq documentation to examine the available LLM models, including their context windows, response quality, speed, and limitations for educational content generation.
- **Evaluate Model Performance:** Compare models based on summarization quality, question generation accuracy, contextual understanding, and response speed for processing study materials.
- **Select the Optimal Model:** Choose llama-3.3-70b-versatile for its balance of high-quality text generation, strong reasoning capabilities, and fast response time, making it suitable for educational content processing and AI-assisted learning.

Activity 1.3: Define the Architecture of the Application

- **Draft an Architectural Diagram:** Create a visual representation of the application architecture, including components such as the React frontend, Flask backend, Supabase database, and Groq API integration.
- **Detail Frontend Functionality:** Outline how users interact with the application by creating an account, uploading study materials, generating summaries, creating flashcards and quizzes, and viewing personalized study schedules and analytics.
- **Outline Backend Responsibilities:** Specify how the Flask backend handles incoming API requests, processes uploaded documents, communicates with the Groq API, and manages interactions with the database.
- **Describe AI Integration Points:** Define how the backend sends study material content to the Groq API, receives generated summaries, quizzes, and flashcards, processes the responses, and returns structured data to the frontend for display.

Activity 1.4: Set Up the Development Environment

- **Install Python and Pip:** Ensure Python 3.8+ is installed on your machine along with pip for dependency management.
- **Create a Virtual Environment:** Set up a virtual environment using venv to isolate project dependencies:

```
D:\projects>python -m venv myenv  
D:\projects>"d:\projects\venv\Scripts\activate"  
(myenv) D:\projects>pip install -r requirements.txt
```

- **Install Flask:** Use pip to install Flask:

```
(myenv) D:\projects>pip install Flask==3.0.3
```

- **Install Groq SDK:** Install the Groq Python library to interact with the Groq API:

```
(myenv) D:\projects>pip install groq==0.11.0
```

- **Install python-dotenv:** Install dotenv support for secure API key management:

```
(myenv) D:\projects>pip install python-dotenv==1.0.1
```

- **Configure the API Key:** Create a .env file in the project root and add your Groq API key:
GROQ_API_KEY=your_groq_api_key_here
- **Set Up Application Structure:** Create the project directory structure including folders for templates, static files (CSS, JS), and main application files (app.py, requirements.txt, run_public.py).

```
▼ static  
  > css  
  > js  
  > templates  
⚙ .env  
📄 .gitignore  
🐍 app.py  
📄 requirements.txt  
🐍 run_public.py
```

Milestone 2: Core Functionalities Development

Activity 2.1: Develop Core Content Generation Features

Implement the core AI features of the StudyAI application as intelligent services driven by study material uploaded by the user.

- **Summary Generator:** Generates concise and structured summaries from uploaded study materials, including topic overviews, key concepts, important definitions, and revision notes.
- **Flashcard Generator:** Produces question-and-answer flashcards from study content, helping students revise important concepts and track their learning progress.
- **Quiz Generator:** Creates AI-generated quizzes in different formats such as Multiple Choice Questions (MCQs), True/False questions, and Short Answer questions, along with explanations and score analysis.
- **Study Schedule Generator:** Generates a personalized 7-day study plan based on uploaded materials and weak topics identified from quiz performance.

Activity 2.2: Implement the Flask Backend

Activity 2.2: Implement the Flask Backend

Define Routes in Flask:

- Set up routes in `app.py` for document upload, summary generation, flashcard generation, quiz generation, study schedule generation, and analytics retrieval.

Process User Input:

- Create React forms and pages for users to upload PDF/DOCX files or paste raw text for AI processing.
- Use Flask's `request.get_json()` and `request.files` to retrieve study material data submitted from the frontend.
- Validate uploaded files, ensure supported file formats, and verify that the extracted content is not empty before processing.

Integrate API Calls:

- Within the API route handlers, construct prompts based on the uploaded study material and the selected AI feature.
- Submit the prompts to the Groq API using `client.chat.completions.create()` with the `llama-3.3-70b-versatile` model.
- Process and validate the AI responses and return structured summaries, flashcards, quizzes, and study schedules to the frontend.

Milestone 3: App.py Development

Milestone 3 focuses on creating and refining the core logic of the app.py file, which serves as the backbone of the CaptionAI application. In this milestone, you'll set up each feature's functionality through Flask routes, establish reliable prompt construction and API interaction, and conduct unit testing to ensure each feature performs as expected.

Activity 3.1: Writing the Main Application Logic in app.py

Define the Core Routes in app.py:

- Establish routes for CaptionAI's two endpoints, each serving a distinct purpose:
 - **/** — Home page route that renders the main index.html interface

```
@app.route("/")
def index():
    return render_template("index.html")
```

- **/generate** — POST endpoint that accepts JSON input, invokes the AI, and returns structured content

```
@app.route("/generate", methods=["POST"])
def generate():
    data = request.get_json()
    product_info = data.get("product_info", "").strip()
    platform = data.get("platform", "instagram").lower()
    tone = data.get("tone", "professional").lower()
    if not product_info:
        return jsonify({"error": "Product information is required."}), 400
    if len(product_info) > 2000:
        return jsonify({"error": "Input too long. Please keep it under 2000 characters."}), 400
    try:
        prompt = build_prompt(product_info, platform, tone)

        chat_completion = client.chat.completions.create(
            messages=[
                {
                    "role": "system",
                    "content": "You are a social media content expert. Always respond with valid JSON only, no markdown formatting."
                },
                {
                    "role": "user",
                    "content": prompt
                }
            ],
            model="llama-3.3-70b-versatile",
            temperature=0.8,
            max_tokens=2048,
```

```
)

response_text = chat_completion.choices[0].message.content
result = extract_json(response_text)

# Validate structure
if not all(k in result for k in ["captions", "hashtags", "ctas"]):
    raise ValueError("Invalid response structure from AI")

return jsonify({
    "success": True,
    "captions": result["captions"],
    "hashtags": result["hashtags"],
    "ctas": result["ctas"],
    "platform": platform,
    "tone": tone
})

except ValueError as e:
    import traceback
    traceback.print_exc()
    return jsonify({"error": f"Failed to parse AI response: {str(e)}"}), 500

except Exception as e:
    import traceback
    traceback.print_exc()
    return jsonify({"error": f"Generation failed: {str(e)}"}), 500
```

Set Up Route Handlers for Each Feature:

- For the /generate route, implement logic that reads product_info, platform, and tone from the incoming JSON body using request.get_json().
- Apply input validation: return a 400 error if product_info is empty or exceeds 2000 characters.
- Route AJAX requests from the frontend to the /generate endpoint, which processes data and returns a JSON response.

Define Prompt Engineering Helpers:

- **TONE_DESCRIPTIONS** dictionary: maps professional, casual, and promotional tones to detailed copywriting instructions embedded in the prompt.

```
TONE_DESCRIPTIONS = {  
    "professional": "formal, authoritative, and business-oriented. Use clear, concise language that conveys expertise and credibility.",  
    "casual": "friendly, conversational, and relatable. Use informal language, emojis, and a warm tone that feels personal.",  
    "promotional": "exciting, persuasive, and action-driven. Use power words, urgency, and compelling offers to drive engagement."  
}
```

- **PLATFORM_TIPS** dictionary: maps instagram and linkedin to platform-specific content guidelines (character limits, style expectations).

```
PLATFORM_TIPS = {  
    "instagram": "optimized for Instagram – visually descriptive, emotionally engaging, with strong visual storytelling. Max 2200 characters.",  
    "linkedin": "optimized for LinkedIn – professional insights, value-driven, thought leadership style. Max 3000 characters."  
}
```

- **build_prompt()**: assembles a structured prompt string from the product info, selected platform tip, and tone description, instructing the model to return strictly valid JSON.

```
def build_prompt(product_info: str, platform: str, tone: str) -> str:  
    tone_desc = TONE_DESCRIPTIONS.get(tone, "engaging")  
    platform_tip = PLATFORM_TIPS.get(platform, "social media")  
  
    return f"""You are an expert social media content strategist and copywriter.
```

Integrate the Groq AI Responses in Each Function:

- Call `client.chat.completions.create()` with a system message enforcing JSON-only responses and a user message containing the assembled prompt.
- **extract_json()**: a robust parser that first attempts `json.loads()`, then strips markdown code fences (````json` blocks), and finally uses a regex to locate a raw JSON object ensuring reliable parsing across model response variations.

```
def extract_json(text: str) -> dict:
    """Extract JSON from LLM response, handling markdown code blocks."""
    # Try direct parse first
    try:
        return json.loads(text)
    except json.JSONDecodeError:
        pass
    # Try extracting from markdown code block
    match = re.search(r'```(?:json)?\s*([\s\S]*)```', text)
    if match:
        try:
            return json.loads(match.group(1).strip())
        except json.JSONDecodeError:
            pass
    match = re.search(r'\{([\s\S]*\)}', text)
    if match:
        try:
            return json.loads(match.group(0))
        except json.JSONDecodeError:
            pass
    raise ValueError("Could not extract valid JSON from response")
```

- Validate that the parsed result contains all three required keys — captions, hashtags, and ctas — before returning it to the frontend.
- **Home route:** `/` → renders `index.html/generate` (POST) → accepts JSON, returns `{success,captions,hashtags,ctas,platform,tone}`

Milestone 4: Frontend Development

Milestone 4 focuses on developing a user-friendly and visually appealing interface for StudyAI. This involves designing a modern and responsive dashboard using React.js and creating dynamic components to display uploaded documents, AI-generated summaries, flashcards, quizzes, and study schedules based on user interactions.

Activity 4.1: Designing and Developing the User Interface

Set Up the Base User Interface:

- Develop the main application interface with pages including Dashboard, Upload Material, Summary, Flashcards, Quiz, Study Schedule, Analytics, and Profile.
- Use reusable React components and semantic HTML elements to keep the application structure organized and accessible.
- Integrate icons and modern typography to provide a clean and intuitive learning experience.

Design a Responsive Layout Using CSS:

- Create responsive styles for all pages using CSS and React component styling to ensure a modern and user-friendly interface.
- Use flexbox and grid layouts to organize dashboards, cards, statistics, and study materials efficiently.
- Add responsive design support to ensure the application works properly on desktop, tablet, and mobile devices.

Create Separate UI Components for Each Feature:

- Document Upload Section: Allows users to upload PDF, DOCX files, or paste raw text for AI processing.
- Summary Cards: Display AI-generated summaries with headings, key concepts, and revision notes.
- Flashcard Interface: Display question-and-answer cards with options to flip, shuffle, and track learning progress.
- Quiz Interface: Display questions, answer options, scores, explanations, and performance analysis.
- Study Schedule Cards: Display personalized day-by-day study plans and recommended tasks.

Activity 4.2: Creating Dynamic Content Rendering with React Handle User Interactions Asynchronously:

- Implement event handlers that send requests to the Flask backend APIs for document upload, summary generation, flashcard creation, quiz generation, and study schedule generation.
- During processing, display loading indicators and disable action buttons until the AI response is received.

Render Results Dynamically:

- Dynamically display uploaded documents, summaries, flashcards, quizzes, analytics, and study schedules using React state management and reusable components.
- Automatically update the interface whenever new AI-generated content is received from the backend.

Restore UI State After Processing:

- Re-enable buttons and restore the original interface after successful or failed API requests.
- Display appropriate success and error messages to provide feedback and improve user experience.

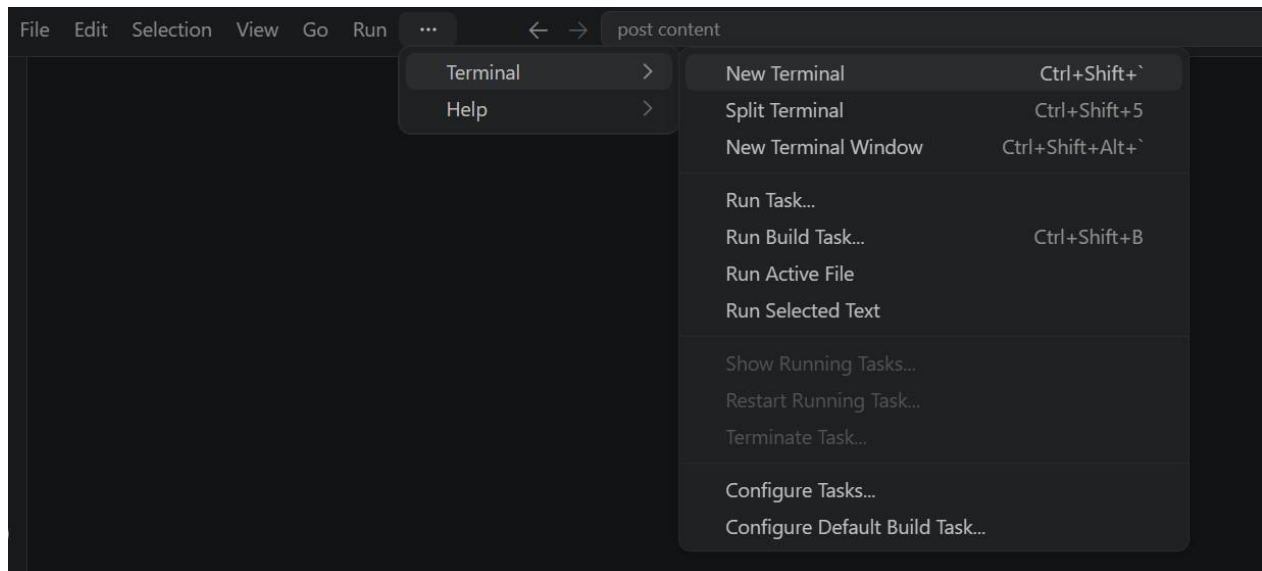
Milestone 5: Deployment

In Milestone 5, the focus is on deploying the StudyAI – Smart AI Study Companion & Quiz Generator application first locally to verify that all features operate correctly, and then publicly by hosting the frontend and backend services. This involves configuring the server environment, setting up database and API credentials, managing dependencies, and making the application accessible to users over the internet.

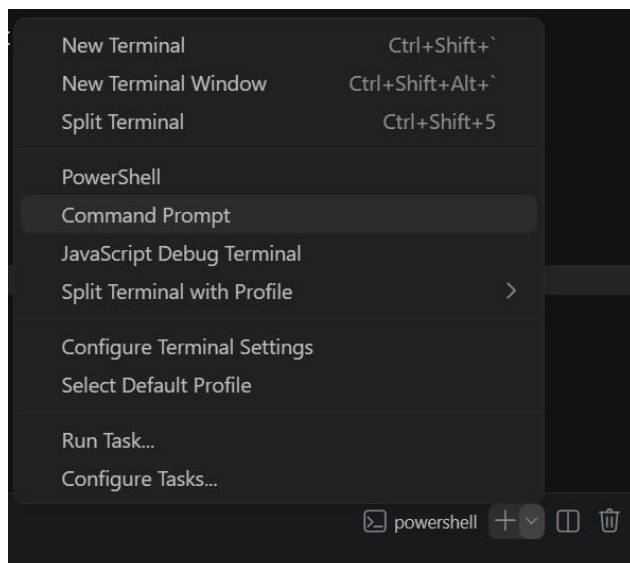
Activity 5.1: Preparing the Application for Local Deployment

Set Up a Virtual Environment:

- Open a New Terminal



- Then open “Command Prompt”



- Create and activate a virtual environment, then install all required dependencies from requirements.

```
D:\projects>python -m venv myenv
D:\projects>"d:\projects\venv\Scripts\activate"
(myenv) D:\projects>pip install -r requirements.txt
```

Configure Environment Variables:

- Create a .env file in the project root and add your Groq API key. The app loads this automatically using python-dotenv at startup:
GROQ_API_KEY=your_groq_api_key_here

Activity 5.2: Local Testing and Verification

Start the Flask Development Server:

- Run the application locally using:

```
(myenv) D:\projects>python app.py
```

- Once running, open a browser and navigate to "http://127.0.0.1:5050" to access the app.

```
(venv) D:\projectsSB\AI in healthcare\startup-validator>python app.py
* Serving Flask app 'app'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment.
d. Follow link (ctrl + click)
* Running on http://127.0.0.1:5050
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 442-110-195
```

- Interact with the caption generator test all three tones (Professional, Casual, Promotional) and both platforms (Instagram, LinkedIn) to verify the AI responses are correctly generated, parsed, and rendered.

Activity 5.3: Public Deployment via Ngrok

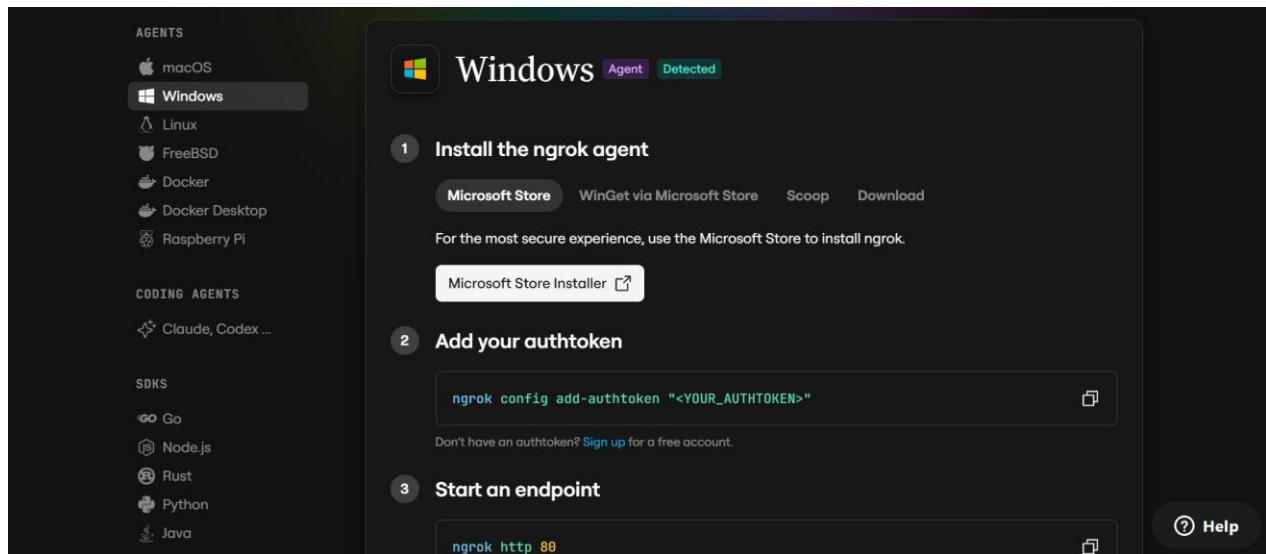
Ngrok creates a secure tunnel from a public URL to your local Flask server, making the app accessible from anywhere without a cloud hosting provider.

Install Ngrok and pyngrok:

- Install the pyngrok Python wrapper, which manages Ngrok programmatically within your Python project:

```
D:\projects>pip install pyngrok
```

- Download and install the Ngrok client from <https://ngrok.com/download>.
- Click on “Microsoft Store Installer”.



Configure Your Ngrok Authtoken:

- Sign up at ngrok.com and copy your authtoken from the Ngrok dashboard.
- The run_public.py script sets the authtoken automatically using:

```
D:\projects>ngrok.set_auth_token("YOUR_NGROK_AUTHTOKEN")
```

- Replace the TOKEN value in run_public.py with your own Ngrok authtoken before running.

Run the Public Deployment Script:

- Instead of running app.py directly, launch run_public.py to start both the Ngrok tunnel and the Flask server:

```
D:\projects>python run_public.py
```

- The script opens an HTTP tunnel on port 5000 and prints the live public URL in the terminal:

```
=====
YOUR PUBLIC WEBSITE URL IS LIVE!
URL: https://xxxx-xx-xx-xxx-xx.ngrok-free.app
=====
```

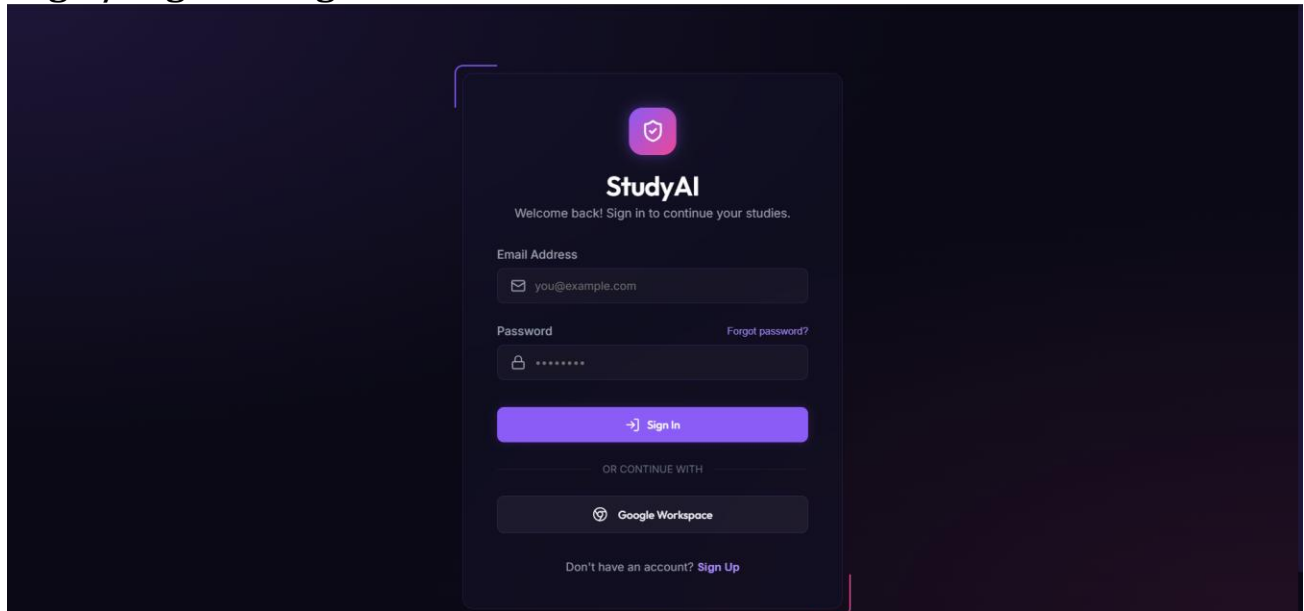
- Share this URL with anyone who can access your CaptionAI app directly from their browser without any installation.

Important Notes:

- **debug=False** is required in run_public.py to prevent Flask from spawning a reloader process, which would interfere with Ngrok's tunnel management.
- The public URL changes each time you restart run_public.py (on the free Ngrok plan). For a persistent URL, upgrade to a paid Ngrok plan and configure a reserved domain.
- Ngrok free tier sessions expire after a few hours of inactivity. Restart run_public.py to get a new URL when this happens.

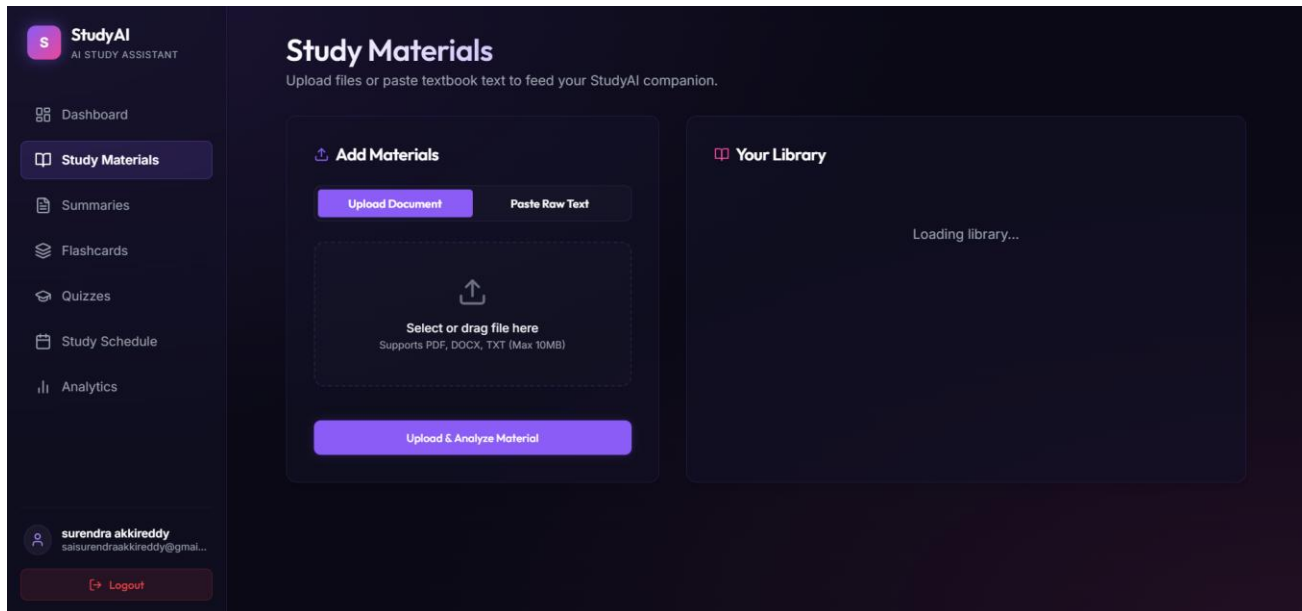
Exploring the Website's Web Pages:

Login/Register Page



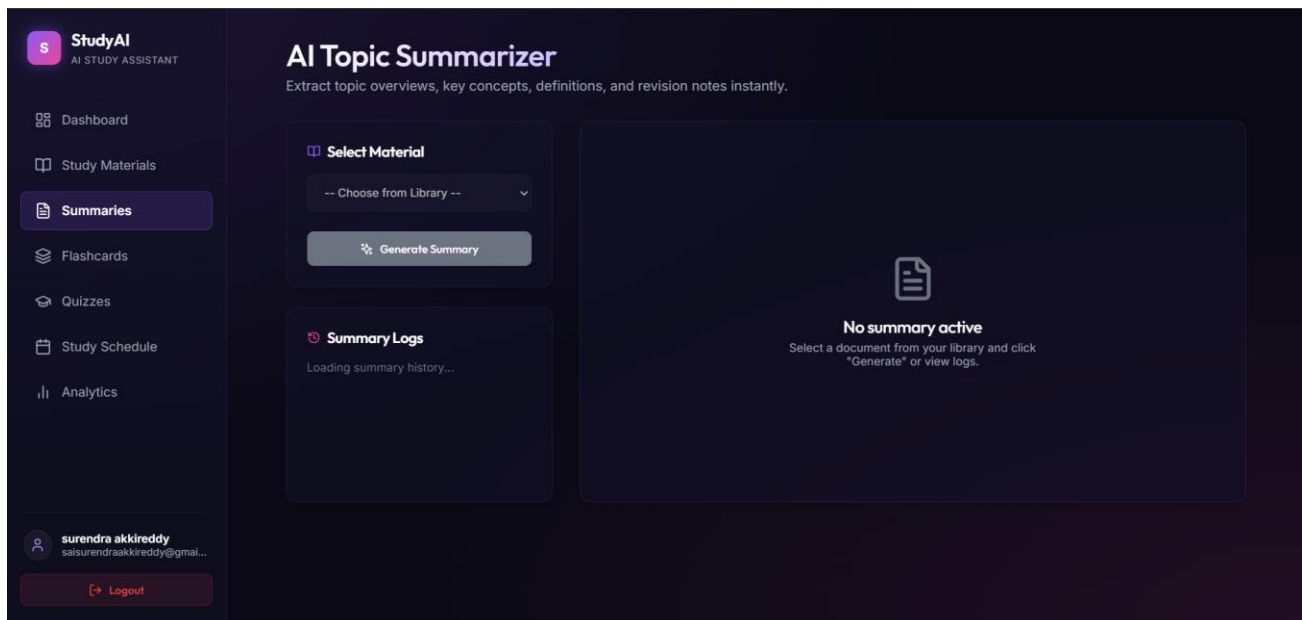
Description: The **Login Page** provides secure access to the StudyAI platform by allowing users to sign in using their registered email and password. It includes input fields for user credentials and validates the entered information before granting access. The page is integrated with Supabase Authentication to ensure secure user verification and session management. It also provides options for new users to register and existing users to recover forgotten passwords. The login interface is designed to be simple, responsive, and user-friendly across different devices. After successful authentication, users are redirected to their personalized dashboard to access all learning features.

Study Material



Description: The **Study Material Upload** page allows users to upload learning resources such as **PDF, DOCX, and text files** into the StudyAI platform. It serves as the entry point for generating AI-powered summaries, flashcards, and quizzes. The page extracts and processes the content of uploaded documents for further analysis. Users can easily manage and access their study materials in one place. The interface is designed to be simple, responsive, and user-friendly for seamless document uploading. This feature helps students organize their study resources and enables personalized learning experiences.

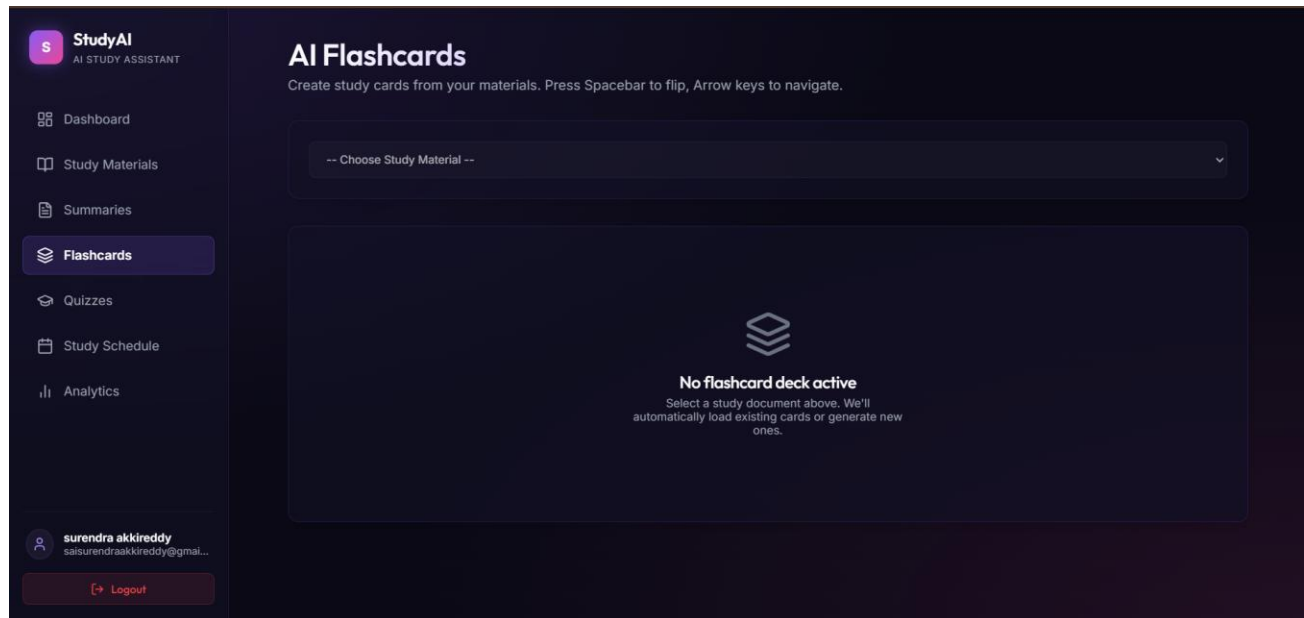
Summaries



Description: The **Summaries Page** allows users to generate concise summaries from uploaded study materials such as PDFs, DOCX files, and text documents. It uses AI technology to extract the most important information and present it in an easy-to-read format. The page helps students quickly understand lengthy content and saves time during exam preparation. Users can view, copy, and save the generated summaries for future reference. It also supports personalized learning by

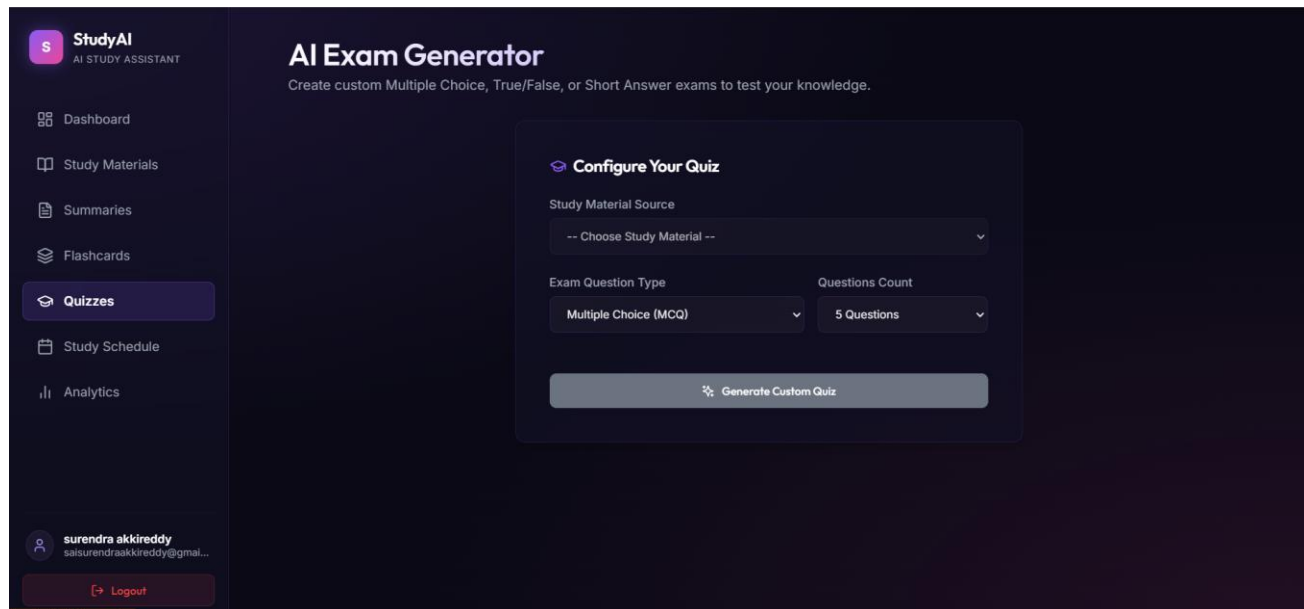
simplifying complex topics into key points. This feature improves learning efficiency and enables students to revise concepts more effectively.

AI FlashCards



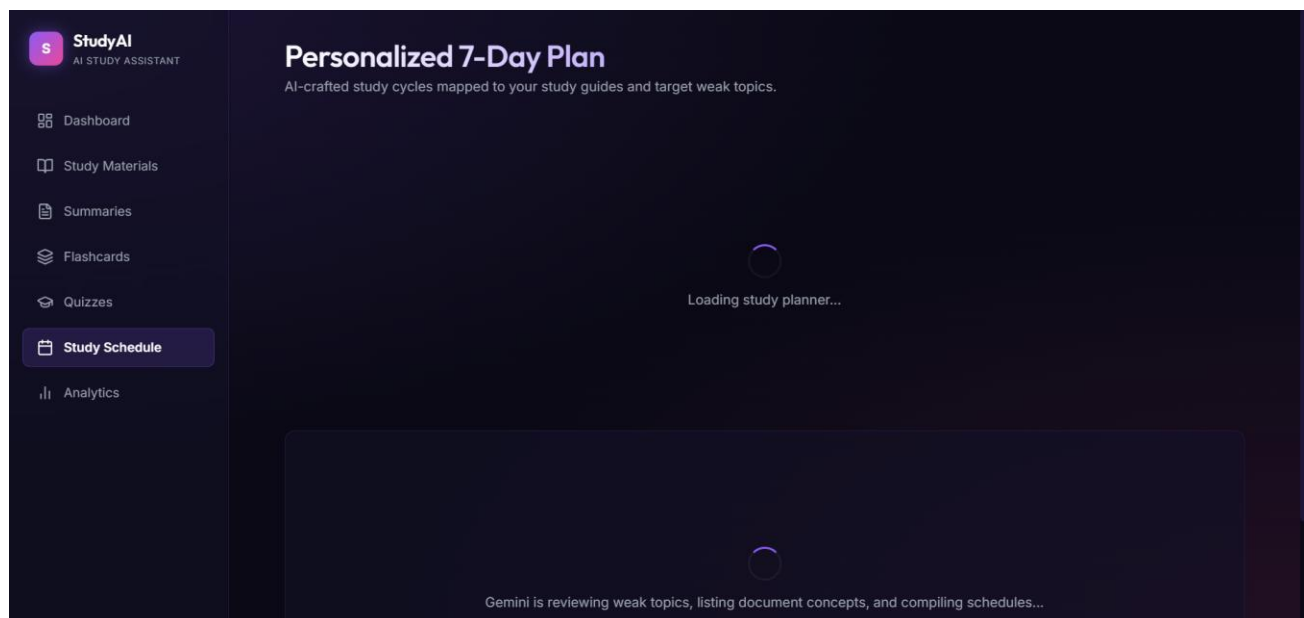
Description: The **AI Flashcards** page helps students learn important concepts through automatically generated question-and-answer cards from uploaded study materials. Users can upload a document and generate flashcards instantly using artificial intelligence. Each card can be flipped to view the answer, making learning more interactive and engaging. The page also includes features such as shuffling cards, marking cards as mastered, and saving flashcard decks for future use. Progress tracking helps students monitor their learning and revise difficult topics effectively. This feature makes studying easier, faster, and more personalized.

Quizzes



Description: The **Quizzes** page allows students to generate AI-powered quizzes from their uploaded study materials. Users can choose the quiz type and the number of questions according to their learning needs. The system automatically creates multiple-choice questions along with correct answers and explanations. Students can attempt the quizzes to test their understanding of different topics. The page also tracks quiz performance and helps identify weak areas that need improvement. This feature makes exam preparation more interactive, personalized, and effective.

Study Schedule



Description: The **Study Schedule** page helps students create a personalized study plan based on their learning goals and available time. It analyzes uploaded study materials and generates an organized timetable for effective exam preparation. The page allows users to track daily study tasks and monitor their progress. It also recommends topics that need more attention based on quiz performance and analytics. The schedule is designed to improve time management and maintain consistent study habits. This feature enables students to learn efficiently and achieve their academic goals in a structured manner.

